Chapter 1

AI Engineering Is Engineering

1. The Engineering Question

Most AI projects start with a technology looking for a place to go. A team gets access to a capable model, someone asks where it could be used, and within a week there is a list of features that could "add AI". Some of those features turn out well. Many turn out to be expensive ways of doing something a simpler component already did, or would have done, at lower cost and with fewer surprises.

The question behind that list is:

> "Where can we use an LLM?"

It sounds practical and it is the wrong starting point, for the same reason "where can we use a message queue?" is the wrong starting point. It begins with the solution and goes looking for problems that fit it.

This book starts from a different question:

What problem are we trying to solve, and what is the simplest technology that solves it well enough?

That question is not an argument against AI. Sometimes the answer is a language model, and sometimes nothing simpler will do. The point is that the answer comes *after* the problem has been stated, the constraints have been named and the alternatives have been compared. Starting from the problem does not rule AI out. It puts AI in the same position as every other technology: a choice that has to earn its place, not a starting assumption.

Language models make this discipline harder to keep, for reasons that will come up again and again in this book:

- **They are easy to call.** Adding a model call often takes a few lines of code. The ease of the call hides the cost of everything around it: latency, model management, evaluation, failure handling.
- **Their output is persuasive.** A fluent answer looks correct. A demo that works on three examples looks like a feature.
- **They are probabilistic.** The same component can do the right thing on one input and the wrong thing on a similar one, and the cause is often not visible in the code.
- **They are partly observable.** You can see what went in and what came out. You usually cannot see why one output was produced instead of another.

None of these properties makes AI engineering a different discipline. They make the ordinary discipline of engineering — state the problem, know what must be true, gather evidence, compare alternatives, decide, and know when to revisit — more necessary, not less.

This chapter's case study is small and real. A simple keyword rule decided whether each request to one of the lab system's entry points should get an answer or a diagram. For a while that was the right design. Then a user attached an image and asked what was in it, and the rule sent the request down the wrong path. The fix replaced the rule with a model call, which added one model call to every request.

The case is not in this chapter because the model won. It is here because it shows the questions this book is about: when a deterministic rule stops being sufficient for a problem, what the alternative costs, and what you know and don't know after making the change.

By the end of the chapter you should be able to:

- state a problem before choosing a technology, and say what "well enough" means for it;
- tell whether a part of your system needs to *match* or to *interpret*;
- list the costs a probabilistic component adds beyond the call itself;
- describe the engineering loop the rest of the book applies, and say which of its stages a given decision actually went through.



2. Core Concepts

Problem-first engineering

A problem statement is the most underrated artifact in an AI project. It answers four questions before any technology is mentioned:

- **What is the task?** What should the system do, for whom, in which situations?
- **What does "well enough" mean?** What quality, latency and cost would make the result acceptable? What failure would make it unacceptable?
- **What must always be true?** Which constraints are not negotiable: who may see which data, which actions are allowed, what must never be sent outside the system?
- **How will we know?** What observation or measurement would tell us the problem is solved?

A team that can't answer these questions isn't ready to choose a technology. It can still prototype, and prototypes are useful for learning what the problem is. But a prototype that "works" is evidence about the prototype, not about whether the problem has been solved.

> *Illustrative.* "Add an AI assistant to the support page" is a solution. "Reduce the number of support tickets that ask for information already in the documentation, without giving users wrong answers about billing" is a problem. Only the second tells you what to measure, what to protect and which alternatives to compare. A better search box, a rewritten documentation page and a language model are all candidates for it.

Problem-first engineering does not mean the problem statement is fixed. Problems change, and the case study in this chapter is about exactly that. When the problem changes, re-examine the technology against the new problem instead of patching the old solution.

The simplest technology that solves the problem well enough

The principle this book returns to is:

Use the simplest technology that solves the problem well enough.

Three words in it carry the weight.

Simplest does not mean "non-AI". It means the option with the least added cost, complexity and uncertainty *for this problem*. Sometimes that is a lookup table or a search index. Sometimes it is a model call, because the alternatives would need so many rules and exceptions that they would be more complex than the model they avoid.

Solves means the option actually does the task, not that it handles the examples someone tried. That claim needs evidence, and gathering it is the subject of Chapter 2.

Well enough means there is a threshold, and it was set before the options were compared. Without one, every option can be made to look adequate or inadequate after the fact.

The principle cuts both ways. It argues **against** adding a model where a rule, a query or a form already solves the problem well enough, and **for** a model where the problem requires something rules cannot express. It doesn't decide anything by itself. It tells you which comparison to make.

Deterministic and probabilistic components

The most useful distinction in this book is between two kinds of components.

A **deterministic component** produces the same output for the same input, by rules someone wrote down, as long as its state, configuration and other relevant conditions stay the same. A keyword check, a database query, a validation function and a permission check are deterministic. You can read the rules, enumerate their cases and, when

the input domain is finite and tractable, test them exhaustively. Deterministic does not mean correct. When a deterministic component is wrong, it is wrong in the same way every time the same input appears, and the cause is in the rules or in how well they fit the problem.

A **probabilistic component** produces outputs from a learned model rather than from rules someone wrote down. A language model is the main example in this book: at each step it produces a probability distribution over possible outputs, and the runtime selects from it (Chapter 3). Probabilistic does not mean random; its outputs are shaped by its input and are often highly consistent. Settings such as temperature can make its output repeatable on a given setup (Chapter 3), but repeatable is not the same as specified: you still can't read off, in advance, what it will do on an input nobody has tried.

Neither kind is better in general. Deterministic components are predictable, cheap to run and straightforward to test, but they only do what their rules say. Probabilistic components may handle inputs nobody anticipated — paraphrases, ambiguity, context, intent — but they can also fail on them, and whether they do is something you find out by measuring, not by reading the code.

The difference that matters for engineering is **what you must do to trust each one**:

	Deterministic component	Probabilistic component
Specification	The rules are the specification	Behavior is specified only by examples and instructions
Testing	Cases can be enumerated from the rules	Behavior must be sampled; tests show what happened on the cases tried
Failure	Repeats on the same input; the cause is in the rules	May depend on wording or context; the cause may not be visible

	Deterministic component	Probabilistic component
Evidence of quality	Reading the rules and testing their cases	Measuring on a representative set of cases (Chapters 10 and 11)
Residual uncertainty	Mostly about whether the rules match the problem	About the rate and kind of errors, which must be estimated

When you replace a deterministic component with a probabilistic one, you don't only change what the system can do. You change what you have to measure, test and observe, and you accept a kind of uncertainty that did not exist before. That is often the right trade. It should always be a visible one.

This is a working engineering distinction, not a taxonomy every component fits cleanly. Real systems mix both kinds: a rule with a model fallback, a model constrained to a fixed set of outputs, a deterministic check on a model's output. The distinction is useful because it points to what must be measured.

Matching versus interpreting

A practical way to tell which kind of component a part of your system needs is to ask whether the task is **matching** or **interpreting**.

Matching asks whether an input contains, equals or satisfies something that can be written down: a word, a code, a format, a value in a range. Rules are good at matching.

Interpreting asks what an input *means*: what the user wants, whether two phrasings say the same thing, whether a passage supports a claim. That is a different problem, even when it looks like matching. Rules can approximate interpretation, often surprisingly well, for as long as the inputs stay close to what the rule's author had in mind.

The trouble starts when a rule is used to approximate interpretation and the inputs change: new features, new users or new kinds of content bring inputs the author never imagined. The rule keeps matching exactly what it was written to match. What it matches no longer

means what it used to. Nothing in the rule has changed, and nothing in the code signals the failure. The case study in this chapter is a clean example.

Complexity is a cost

Every option in a design comparison has costs; rules have to be maintained, and asking the user adds interface. A probabilistic component brings costs that are easy to overlook, because the call itself is easy to write. It can add:

- **latency:** another call to wait for, often on every request;
- **cost:** compute or per-call charges;
- **operational dependencies:** a model to host and version, or a provider to depend on;
- **failure modes:** timeouts, unavailable models, malformed outputs, wrong outputs that look right;
- **observability requirements:** recording inputs, outputs and decisions so failures can be diagnosed;
- **evaluation requirements:** a way to know whether the component is good enough, and whether it got worse;
- **uncertainty:** an error rate that must be estimated rather than read from the code.

None of these is a reason to avoid AI. They belong in the comparison next to the costs of the alternatives, before the decision is made, not discovered after it ships. Chapter 12 treats latency and cost as measured design inputs, and Chapter 14 shows how to combine them with quality and constraints in one decision.

The engineering method

This book applies one loop to every decision:

Problem → Hypothesis → Baseline → Experiment → Measure → Compare → Decide → Observe → Iterate

In brief: state the **problem** and what "well enough" means; turn a belief about what will solve it into a **hypothesis** a specific observation could prove wrong; establish a **baseline** of how the system behaves now; run an **experiment** that changes one thing; **measure** what happened and under which conditions; **compare** the result with the

baseline and the threshold, costs included; **decide**, and record why; **observe** the decision in use; and **iterate** when it no longer fits.

The loop matters for three reasons.

It puts the decision after the evidence. Implementing first and measuring afterwards turns every result into a justification for something already built.

It makes gaps visible. Real decisions often skip stages: no baseline, an experiment with one case, nobody observing. Named stages let you say which ones were skipped, which is the first step toward knowing how much a decision can be trusted.

It does not end. A decision made well for one problem can become wrong when the problem changes, without any change to the code.

This chapter establishes why the loop matters, and its case study shows the whole loop on a small decision, including the stages it did not go through. How to run the middle of the loop is Chapter 2's subject; Chapter 14 returns to "decide".

The lab system

The evidence in this book comes from one system, called the **lab system** throughout. It is introduced here once. In later chapters it appears in Lab Notebook entries, not as the subject of chapters.

The lab system is a multi-tenant document question-answering platform. Users ask questions about their organization's documents. The system retrieves relevant passages from that tenant's documents, a language model generates an answer with citations, and a second model call checks whether the answer is supported by the passages. It runs local models on a CPU-only host, with an optional hosted-model fallback for cases the local path cannot answer. Its prompts are in Portuguese, and the book translates them.

The lab exists to provide **bounded, real engineering evidence**: decisions that were actually made, measurements that were actually taken, failures that actually happened, recorded in decision records and investigation records that the Lab Notebook entries cite as sources. It is not a reference architecture and not a recommendation. The technologies it happens to use appear only as context for its experiments, never as requirements.

Every lab result in this book is bounded by its conditions: one system, specific models and versions, one kind of hardware, specific task sets and sample sizes, often small. Evidence labels keep those bounds visible. This chapter uses three of the five labels defined in Chapter 2: *Observed in the lab*, *Not measured in this project* and *Illustrative*.

3. Options and Trade-offs

When part of a system has to decide something about its input — which path a request takes, which category a message belongs to, whether a document is relevant — there are several broad ways to build that decision. None is always right.

Option	What it buys	What it costs	Main risk
A. Fixed rules (keywords, patterns, lookup tables)	Predictable, fast, cheap, testable by reading the rules	Rules to maintain; only handles what its author anticipated	Wrong in the same way, every time, on inputs the rules didn't anticipate
B. Ask the user (separate entry points, explicit choices, forms)	No inference needed; the user states the intent	More interface; users must know which option they want	Users choose wrongly or don't want to choose
C. Rules plus patches (add exceptions as failures appear)	Keeps the deterministic component; fixes the reported case	Each patch fixes one input, not the class of inputs	The rule grows until no one can predict it; the next failure is not prevented
D. A probabilistic classifier (a model call, or a small trained classifier)	Can interpret meaning, paraphrase and context	Latency, cost, dependencies, failure modes, evaluation	An error rate that has to be measured and can't be read from code

Option	What it buys	What it costs	Main risk
E. Rules first, model for the rest	Model cost paid only where rules are unsure	Two mechanisms to maintain and test, and a boundary between them	The rule still decides confidently and wrongly on inputs it misreads

Each option is valid for some problem. **Fixed rules are often the right first answer**: a rule that is simple, fast and correct on the inputs the system actually receives is hard to beat. **Patching rules indefinitely** feels cautious, but each patch fixes one input rather than the class of failure, and the rule becomes harder to predict over time. **A probabilistic classifier changes the evidence required**: "is the decision correct?" stops being a question you answer by reading code and becomes one you answer by measuring.

For and against here mean what they mean throughout the book. The table argues *for* starting with rules where they suffice, which does not mean recommending them everywhere. It argues *against* patching rules indefinitely, which does not mean rejecting rules. The choice depends on the problem, and the problem can change.

4. Lab Notebook

The entries below use the Lab Notebook format defined in Chapter 2. The lab's routing prompt and its labels are translated from Portuguese.

A keyword rule that was right until the problem changed

> **LAB NOTEBOOK 1.1 — From keyword routing to model classification**
>
> **Evidence status** Observed in the lab
>
> **Source** Decision records for the single entry point with keyword routing (ADR 0006, including its later update) and for model-based routing (ADR 0024); image attachment on questions (ADR 0023)
>
> **Context** The lab system's single entry point accepts either a question or a request for a diagram, and must decide which path each request takes. The

first design used a fixed keyword list: the request text was checked, after removing accents, for words associated with requesting a visual (for example “diagram”, “flow”, “architecture”, “chart” and “imagem”, Portuguese for “image”). Any match sent the request to diagram generation; otherwise it went to answer generation. Later, a separate change allowed users to attach an image to a question.

Metadata recorded / missing Decision records and code history recorded (keyword routing introduced 2026-07-24; model-based routing 2026-07-29). The classification model is not named in the record beyond “the resolved chat model”. Runtime version and hardware not recorded in the source.

Question / Hypothesis Not a planned experiment. The question the incident raised was: is the keyword rule still sufficient for routing, now that users can attach images and ask about them?

Method A real user’s request failed. The failing question was reproduced with an attached test image, before and after the change.

Sample size One real incident; one reproduction of the failing question before and after the change.

Result *Observed:* (1) The original record described routing as a plain substring check rather than an extra model call, so that it added no latency and no extra failure mode; the later record describes it as deliberately chosen over a model call for zero added latency. The original record also predicted its weakness: a question that happened to contain a matching word would be routed to diagram generation even when a text answer was wanted. It accepted that trade-off for the first version and named the replacement to consider if false-positive routing became a real problem: a cheap classification prompt or a small local classifier. (2) After image attachment shipped, a real user attached a photo and asked “O que tem nessa imagem?” (“What’s in this image?”). The question contained “imagem”, one of the keywords. The request was routed to diagram generation and returned a diagram built from unrelated retrieved documents instead of a description of the photo. (3) The keyword list was removed. Routing now makes a classification call at temperature 0 that asks the model to answer with exactly one of two words, one for diagram and one for answer. The prompt states that incidental words such as “image”, “diagram” or “architecture” do not by themselves indicate a request for a diagram, and that questions about an attached image’s content should be answered. (4) In the reproduction, the same question with an attached test image produced a diagram before the change and a description of the image after it. (5) The single entry point now makes **one additional model call on every request**. The original single-purpose endpoints, which never routed through this check, were unaffected.

Interpretation The keyword rule did not break. It kept matching exactly what it was written to match. The problem changed: once users could attach

> images, the word “imagem” stopped indicating a request for a picture and started appearing in the most natural question about an attachment. The task had always been interpreting intent; for a while, matching words approximated it well enough. A new kind of input made the approximation fail, and the record’s own revisit condition was met.
>
> **Decision** Replace keyword routing with a model classification call, accepting one additional model call per request in exchange for routing on intent rather than on incidental words. The routing call goes through the same failure-handling path as the system’s other model calls, so a model outage affects routing the same way it affects generation. A regression test pins the exact failing question.
>
> **Limitations** One incident and one reproduction; this is a recorded engineering decision, not a measurement. The classifier is not perfect: the decision record itself notes that it can misjudge an unusually ambiguous question. *Not measured in this project:* routing accuracy before or after the change, on any set of questions; the latency impact and the cost impact of the added call.

Two things in this entry are easy to miss.

The first design was not a mistake. It chose the simplest technology that solved the problem at the time, and recorded both its predicted failure and what to do if that failure became real.

What changed was the input, not the code. Image attachment was a separate feature with its own decision record, and nothing in it touched routing. But it changed the inputs routing received. Features interact through the inputs they create, not only through the code they share.

What the change left unknown

The decision traded one kind of uncertainty for another. The keyword rule had a known, guaranteed failure class: any question containing one of its words went to diagrams, every time. The classifier may misjudge some questions, but which ones, and how often, is not visible from the code. The decision record calls this a qualitatively different risk. The change addressed the observed failure and routed the reproduced case correctly; the decision was accepted despite the measurement gap below.

> **LAB NOTEBOOK 1.2 — Routing accuracy (Not measured variant)**
>
> **Evidence status** Not measured in this project
>
> **Question** How accurately does each routing approach send requests to the intended path, and what does the change cost in latency and compute?
>
> **What would answer it** A labeled set of real requests covering both intents, including questions about attachments, diagram requests phrased without keywords, and questions that mention diagram-related words incidentally. Both approaches run on the same set, with accuracy reported per category and the routing step's latency measured on the same host.
>
> **What we have instead** The single incident and reproduction in Notebook 1.1, the original record's prediction of the keyword rule's failure class, and a regression test for the failing question. Chapter 5 (Notebook 5.3) records the same gap.
>
> **Risk of deciding without it** The decision rests on the nature of the failure, a guaranteed misroute on a common question, rather than on a measured improvement. It does not show that the classifier routes better overall, how often it misroutes, or what the added call costs. A genuine diagram request sent to the answer path would not be detected by anything described here.

Many real decisions are made this way: a single clear failure, a fix that addresses its cause, and measurement that never happens because the visible problem went away. Chapter 10 shows what the measurement would involve.

The case through the loop

Stage	What happened	How complete
Problem	One entry point for two kinds of request; route each correctly	Stated in the first decision record
Hypothesis	The first design: keywords are specific enough to requesting a visual. Later: a classifier will route on intent where words mislead	Stated, with the predicted failure of the first

Stage	What happened	How complete
Baseline	The keyword rule's behavior on the failing question	One case, not a rate
Experiment	The failing question reproduced with an attached image, before and after	One case
Measure	Diagram before, description after	Observed, not measured across a set
Compare	The known failure class against an unmeasured error rate; one added call per request	Qualitative; costs not measured
Decide	Replace the rule; accept the added call	Recorded, with the trade-off stated
Observe	The first design's revisit condition was met by a real user's request	The trigger worked
Iterate	The decision was revisited, and a regression test added	Done once; accuracy not tracked

The table is not a criticism of the decision. Some stages are strong, some rest on a single case and one is missing. Knowing which is which is what lets you say how far the decision can be trusted.

5. Decision Guide

Before introducing an AI component

1. What is the problem, stated without naming a technology?
2. What threshold defines "well enough", and what latency and cost can you afford?
3. What must always be true?
4. What is the simplest option that might clear the threshold?

5. Does the task require matching or interpreting?
6. What would the AI component add (Section 2, “Complexity is a cost”)?
7. What evidence will tell you it works?

Signs that a deterministic rule is no longer enough

- The rule matches words to infer meaning, and the words have started to appear for other reasons.
- A new feature has changed the inputs the rule receives.
- Each failure is fixed by adding an exception.
- The rule fails every time on a common, natural input.
- The revisit condition you recorded has been met.

When adding a probabilistic component, record

- what it buys: the failure class it removes or the capability it adds;
- what it costs, measured or explicitly marked as not measured;
- what becomes uncertain, and which errors would go unnoticed;
- what happens when it is unavailable or returns something unusable;
- what evidence you have, and with which label;
- what would make you revisit the decision.

6. Limits

This chapter establishes a way of thinking. Its evidence is deliberately small.

One observed case, not a benchmark. The case study is a single real failure and a single reproduction. It does not show that classifiers route better than keyword rules in general, or in this system overall. The only quantitative statement it supports is architectural: one additional model call per request. Routing accuracy, latency impact and cost impact are *not measured in this project* (Notebook 1.2).

One system, one language setting. The lab’s keyword list mixed Portuguese and English words, and its routing prompt is in Portuguese. Whether the same rule would have failed, or the same classifier would have worked, in another language or another system is not known.

The principle is not a formula. "The simplest technology that solves the problem well enough" tells you which comparison to make. It does not make the comparison, set the threshold or measure the options. Chapters 2, 10 and 14 cover those steps.

7. Going Further

This chapter argued why an AI engineer should think this way. It did not show how to produce the evidence its own case study lacks: a baseline worth comparing against, an experiment that could refute a fix, a measurement that changes a decision.

Chapter 2 — Evidence Before Opinion: Running an Engineering Experiment answers that question. It turns the middle of the loop — hypothesis, baseline, experiment, measure, compare — into a practical method, introduces the five evidence labels and the Lab Notebook format, and uses the lab system to show three proposed fixes measured before any was implemented.

The chapters after it apply the same stance to each part of an AI system:

- **Part II — Models** (Chapters 3 and 4): what a language model gives you and what it doesn't, and how to choose one.
- **Part III — Shaping Behavior and Knowledge** (Chapters 5 to 8): prompting, embeddings and retrieval, RAG as an engineering decision, and context engineering. Chapter 5 returns to the routing prompt from this chapter's case study.
- **Part IV — Systems That Act** (Chapter 9): tool calling and first agents.
- **Part V — Knowing Whether It Works** (Chapters 10 and 11): evaluating AI systems, and deciding when a benchmark deserves trust.
- **Part VI — Trade-offs** (Chapters 12 to 14): latency and cost as design inputs, security fundamentals, and making the decision.

Book 2 — Production AI Engineering covers running AI systems in production. **Book 3 — AI Model Lab** covers models and evaluation science in depth.